

Backend Module

Full-Stack Development

What a Backend Actually Does

The backend is the part of an application that runs on a server rather than in the user's browser — it handles business logic, talks to the database, enforces security rules, and exposes an API the frontend calls to read and write data. Every backend, regardless of language or framework, does some combination of: receiving a request, validating it, doing work (often involving a database), and sending back a response. Understanding this request-response cycle clearly is more valuable than memorizing any one framework's syntax, because it's the same shape in Node.js, Django, Spring Boot, or .NET.

REST API Design

REST (Representational State Transfer) is a set of conventions for designing predictable, resource-oriented APIs. The core idea: model your API around nouns (resources, like `/users` or `/orders`), and use HTTP methods as the verbs that act on them.

- GET — retrieve a resource or list of resources, without changing anything.
- POST — create a new resource.
- PUT/PATCH — update an existing resource (PUT typically replaces it fully, PATCH updates part of it).
- DELETE — remove a resource.
- Good REST design also means using URLs for identity (`/users/42`) and HTTP status codes to communicate outcome, rather than stuffing everything into the response body.

Status Codes & Error Handling

HTTP status codes are the first thing a client checks, and using them correctly makes an API far easier to consume. The broad families are: 2xx for success (200 OK, 201 Created, 204 No Content), 4xx for client errors — the caller did something wrong (400 Bad Request for invalid input, 401 Unauthorized for missing/invalid auth, 403 Forbidden for valid auth but insufficient permission, 404 Not Found), and 5xx for server errors — something broke on the backend itself (500 Internal Server Error). A well-designed backend also returns a consistent, structured error body (not just a status code) so the frontend can show a useful message rather than a generic failure.

Middleware

Middleware is code that runs between a request arriving and the final route handler processing it — a pipeline each request passes through in order. Common uses include: parsing the incoming request body into usable data, logging requests for debugging, authenticating the caller and attaching their identity to the request, and catching errors thrown anywhere downstream so the server can respond gracefully instead of crashing. Thinking of a backend as a pipeline of small, focused middleware functions — rather than one large function doing everything — is what keeps larger backends maintainable.

Server Architecture Basics

Most backends follow a layered structure to keep concerns separated: a routing layer maps incoming URLs/methods to handler functions, a controller/handler layer contains the logic for each specific request, a service/business-logic layer holds reusable rules independent of HTTP (so the same logic can be reused outside a web request, e.g. in a background job), and a data-access layer talks to the database. Keeping these layers distinct — rather than putting database queries directly inside route handlers — is what makes a backend testable and easy to change later without breaking unrelated features.

Building a Production-Style API

A production-quality API adds several things a quick prototype usually skips: input validation on every endpoint (never trust client-supplied data), consistent error responses, environment-based configuration (so the same code runs against different databases/settings in development vs. production without code changes), and automated tests covering the main success and failure paths of each endpoint. These aren't optional polish — they're what separates code that works on your machine from code a team can safely build and deploy on top of.

Learn More

MDN Web Docs — HTTP Overview

<https://developer.mozilla.org/en-US/docs/Web/HTTP>

Node.js Official Documentation

<https://nodejs.org/en/docs>

GeeksforGeeks — REST API Design

<https://www.geeksforgeeks.org/rest-api-introduction/>